Taski · Parte 9: Galería de Evidencias Multimedia

Actividad Guiada + Actividad Final Integradora · Flutter, PostgreSQL y Supabase Storage

Bucket privado `task-attachments` · Tabla `public.task_attachments` · Políticas RLS · Bloqueo Optimista

🔗 **Repositorio GitHub del Proyecto (Actualizado con Parte 9): https://github.com/RoierSinH/Taski**

📹 **Enlace Directo al Video de Demostración:**
https://github.com/RoierSinH/Taski/blob/main/Video_Evidencia_Taski_Parte_9.webm

1. Resumen de la Implementación (Actividad Guiada y Actividad Final)

- **Configuración y Seguridad en Supabase:** Creación del bucket privado `task-attachments` (límite de 15 MB y lista blanca MIME) y la tabla `public.task_attachments` con restricciones `CHECK` (tamaño, versión ≥ 1, longitud de nombre/descripción y validación de prefijo `<user_id>/<task_id>/...`). Aplicación de **7 políticas RLS** (4 en SQL y 3 en `storage.objects`).
- **Flujo CRUD Multimedia Completo:** Subida binaria + registro SQL con compensación ante fallos (`uploadBytes`), consulta por tarea (`listForTask`), generación de URLs firmadas temporales por 300 segundos (`signedUrl`), edición de metadatos con bloqueo optimista por `version` (`updateMetadata`), reemplazo seguro de archivo limpiando el objeto anterior (`replaceBytes`) y eliminación con confirmación (`deleteAttachment`).
- **Galería de Evidencias y Mejoras de Usabilidad:** Vista en cuadrícula con **miniaturas reales para fotografías**, insignias de formato y versión (`v1`, `v2`, `v3`), **contadores dinámicos por tipo**, filtros rápidos, búsqueda, ordenamiento y protección de integridad al intentar eliminar una tarea con adjuntos (Sección 16).

2. Respuestas a las 4 Preguntas de Comprensión (Página 24)

1. ¿Cómo se relaciona una evidencia con una tarea?

Una evidencia se relaciona con una tarea mediante una arquitectura híbrida que combina **base de datos relacional (PostgreSQL)** y **almacenamiento de objetos (Supabase Storage)**:

- En PostgreSQL, la tabla `public.task_attachments` posee la clave foránea `task_id bigint not null references public.tasks(id) on delete cascade`, estableciendo una relación **uno a muchos (1:N)** donde cada tarea puede tener múltiples evidencias asociadas.
- En Supabase Storage, cada archivo físico se guarda dentro del bucket privado `task-attachments` bajo la ruta jerárquica `<user_id>/<task_id>/<uuid>.<ext>`, cuya ruta exacta queda registrada en la columna `storage_path`. Además, las restricciones `CHECK` (`attachment_path_owner` y `attachment_path_task`) verifican que el primer segmento coincida con `user_id` y el segundo con `task_id`.

2. ¿Por qué el bucket es privado?

El bucket `task-attachments` se configura como **privado (`public = false`)** porque las evidencias de una tarea (fotografías, reportes PDF, notas de voz y videos) son información privada del usuario propietario de la tarea. Si fuera público, cualquier persona que conociera o adivinara el `storage_path` podría descargar los archivos sin autenticación. Al ser privado, toda lectura o escritura pasa obligatoriamente por las políticas **Row Level Security (RLS)** de `storage.objects` que verifican `auth.uid()`, y la app consume los archivos mediante **URLs firmadas temporales (`createSignedUrl(path, 300)`)** que expiran a los 5 minutos.

3. ¿Qué diferencia existe entre editar una descripción y reemplazar el contenido?

- **Editar una descripción (o el nombre visible):** Es una operación **exclusivamente sobre metadatos en SQL** (`updateMetadata`). No modifica ni vuelve a subir bytes a Supabase Storage; únicamente ejecuta un `UPDATE` en `public.task_attachments` actualizando `original_name`, `description` e incrementando `version = version + 1` mediante bloqueo optimista (`.eq('version', attachment.version)`). El `storage_path` permanece intacto.
- **Reemplazar el contenido:** Es una operación **compuesta entre Storage y SQL** (`replaceBytes`): primero valida y sube el nuevo binario a una nueva ruta en el bucket, luego actualiza la fila SQL con el nuevo `storage_path`, `mime_type`, `size_bytes` y `version + 1`, y finalmente elimina del bucket el objeto binario anterior. Si el `UPDATE` SQL falla, ejecuta una compensación borrando el nuevo archivo recién subido.

4. ¿Qué ocurre si falla Storage después de una operación SQL?

Como PostgreSQL y Supabase Storage no comparten una transacción distribuida atómica (2PC), se aplican órdenes seguros y pasos de compensación:

- **En el alta (`uploadBytes`):** Primero se sube a Storage y luego se inserta en SQL. Si el `INSERT` SQL falla, el bloque `catch` elimina inmediatamente el objeto subido en Storage (`bucket.remove([objectPath])`), evitando archivos huérfanos.
- **En el reemplazo (`replaceBytes`):** Primero se sube el archivo nuevo, luego se actualiza SQL y al final se elimina el archivo antiguo de Storage. Si falla la eliminación final del archivo antiguo, la fila SQL ya apunta válidamente al nuevo archivo (consistencia funcional preservada).
- **En el borrado (`deleteAttachment`):** Primero se borra la fila en SQL y luego se remueve el objeto en Storage. Si Storage falla tras el borrado SQL, el adjunto deja de aparecer en la interfaz del usuario y se notifica para limpieza del objeto binario.

3. Explicación Técnica de 3 Fragmentos de Código

Fragmento 1 — Interfaz de Usuario: Tarjetas de Galería con Miniaturas e Insignias (`TaskAttachmentsPanel`)

```
Widget _buildMediaPreviewHeader(TaskAttachment a) {
  final color = _colorForAttachment(a);
  if (a.isImage && _thumbnailUrls.containsKey(a.id)) {
    return Stack(
      fit: StackFit.expand,
      children: [
        Image.network(
          _thumbnailUrls[a.id]!,
          fit: BoxFit.cover,
          errorBuilder: (_, __, ___) => Container(
            color: color.withValues(alpha: 0.12),
            child: Icon(Icons.image, size: 42, color: color),
          ),
        ),
        Positioned(top: 8, left: 8, child: _buildFormatBadge(a, color)),
      ],
    );
  }
  return Container(
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: [color.withValues(alpha: 0.18), color.withValues(alpha: 0.06)],
      ),
    ),
    child: Stack(
      alignment: Alignment.center,
      children: [
        Icon(_iconForAttachment(a), size: 46, color: color),
        Positioned(top: 8, left: 8, child: _buildFormatBadge(a, color)),
      ],
    ),
  );
}
```

Explicación: Renderiza el encabezado visual de cada tarjeta en la Galería de Evidencias. Si el adjunto es una fotografía (`a.isImage`), muestra su miniatura real descargada mediante URL firmada temporal (`_thumbnailUrls[a.id]`); si es PDF, audio o video, presenta un ícono distintivo por formato junto con un badge que indica el tipo de medio y su versión actual (ej. `Fotografía · v1`, `PDF · v2`, `Video · v3`).

Fragmento 2 — Acceso a Datos y Consistencia: Reemplazo con Bloqueo Optimista y Compensación (`replaceBytes`)

```
Future<void> replaceBytes({
  required TaskAttachment attachment,
  required Uint8List newBytes,
  required String fileName,
}) async {
  final uid = userId;
  final info = inspectBytes(bytes: newBytes, fileName: fileName);
  final ext = p.extension(fileName).toLowerCase();
  final newPath = '$uid/${attachment.taskId}/${generateUuidV4()}$ext';
  final bucket = client.storage.from(bucketName);

  await bucket.uploadBinary(newPath, newBytes, fileOptions: FileOptions(contentType: info.mime));
  try {
    await client.from('task_attachments').update({
      'storage_path': newPath,
      'original_name': p.basename(fileName).trim(),
      'mime_type': info.mime,
      'size_bytes': info.bytes,
      'version': attachment.version + 1,
    }).eq('id', attachment.id).eq('version', attachment.version).select('id').single();
  } catch (_) {
    try { await bucket.remove([newPath]); } catch (_) {}
    rethrow;
  }
  try { await bucket.remove([attachment.storagePath]); } catch (_) {}
}
```

Explicación: Coordina el reemplazo seguro entre Supabase Storage y PostgreSQL. Sube primero el nuevo objeto binario a una ruta única con UUID v4, luego ejecuta el `UPDATE` SQL filtrando por `id` y por `version` (bloqueo optimista). Si la actualización SQL falla, revierte la subida borrando `newPath`; si tiene éxito, elimina el archivo antiguo (`attachment.storagePath`).

Fragmento 3 — Validación y Seguridad: Inspección de Tamaño/MIME (`inspectBytes`)

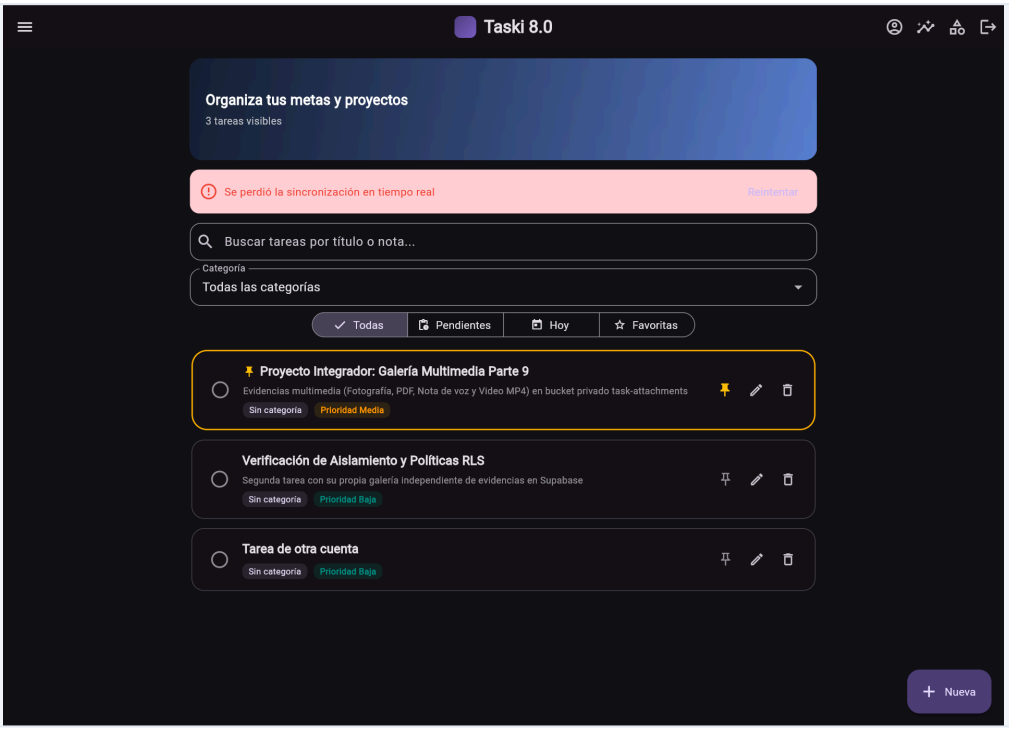
```
static ({String mime, int bytes}) inspectBytes({
  required Uint8List bytes,
  required String fileName,
}) {
  final length = bytes.length;
  if (length < 1 || length > maxBytes) {
    throw StateError('Archivo vacío o mayor de 15 MB');
  }
  final mime = lookupMimeFromPath(fileName);
  if (mime == null || !allowedMime.contains(mime)) {
    throw StateError('Formato no permitido: ${mime ?? p.extension(fileName)}');
  }
  return (mime: mime, bytes: length);
}
```

Explicación: Valida localmente antes de cualquier llamada de red que el archivo tenga entre `1 byte` y `15 MB` (`15 * 1024 * 1024`) y que su tipo MIME pertenezca estrictamente a la lista blanca permitida (imágenes JPG/PNG/WebP, documentos PDF, notas de voz M4A/MP3/WAV y videos MP4), reforzando las restricciones del bucket y las políticas RLS del servidor.

4. Evidencias Visuales de Funcionamiento (Capturas Reales)

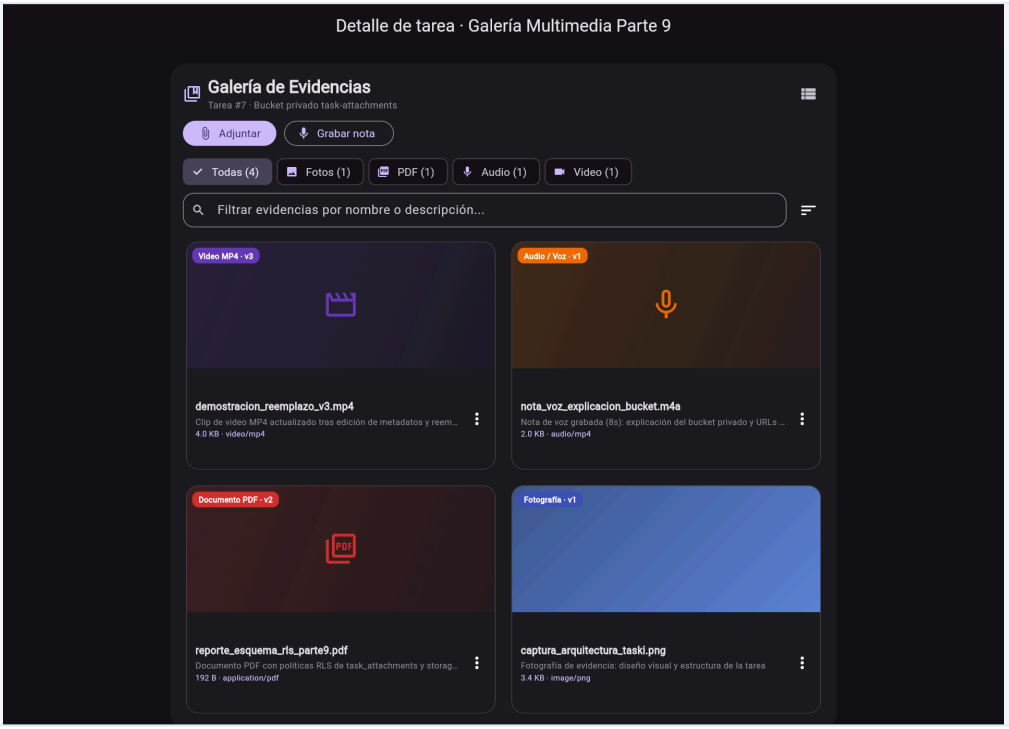
Evidencia 1 **Pantalla Principal con 2 Tareas Independientes**

Vista principal de Taski mostrando las dos tareas de prueba: **"Proyecto Integrador: Galería Multimedia Parte 9"** y **"Verificación de Aislamiento y Políticas RLS"**.



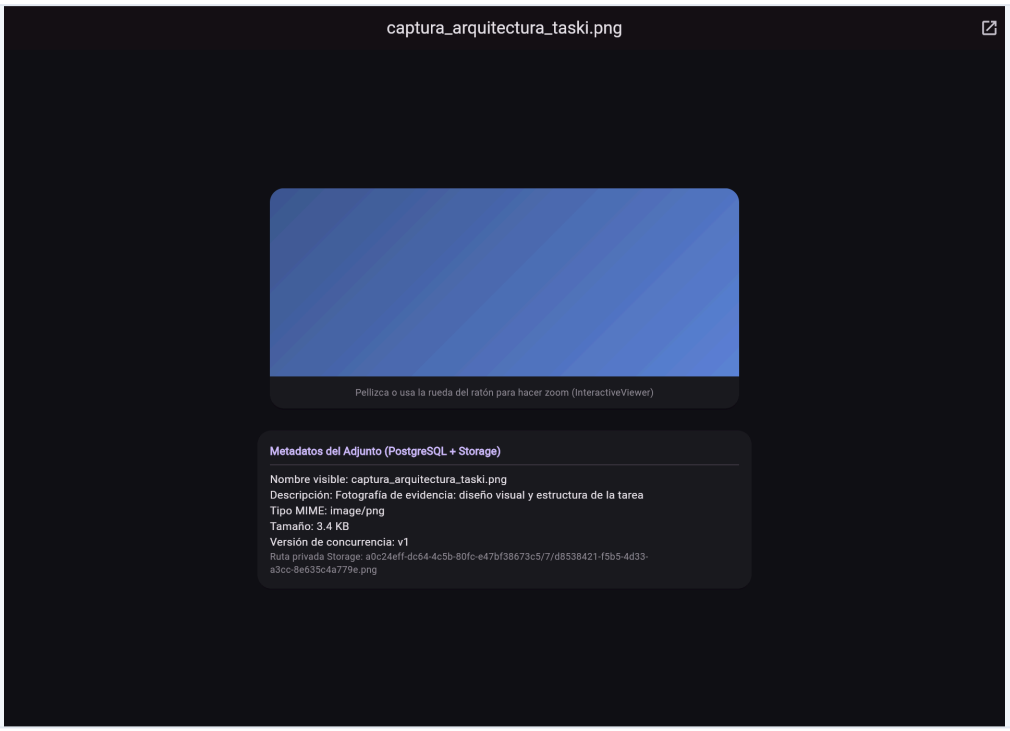
Evidencia 2 Galería de Evidencias con los 4 Tipos de Archivo, Miniatura Real y Contadores

Galería de la Tarea #7 mostrando las 4 tarjetas: **Fotografía PNG (v1)** con miniatura real mediante URL firmada, **Documento PDF (v2)** editado, **Nota de Voz M4A (v1)** y **Video MP4 (v3)** reemplazado, junto con los chips contadores por tipo (`Todas (4)`, `Fotos (1)`, `PDF (1)`, `Audio (1)`, `Video (1)`).



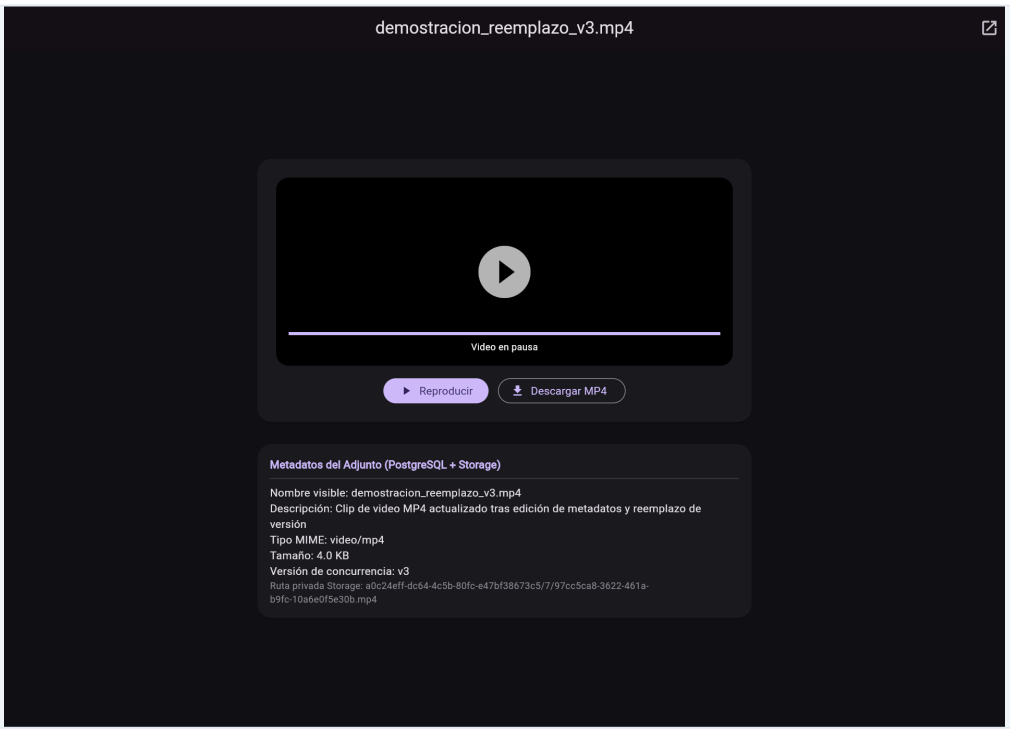
Evidencia 3 Visor Multimedia de Fotografía con Zoom (InteractiveViewer) y Metadatos

Apertura de la evidencia fotográfica mediante `createSignedUrl(path, 300)` con soporte de zoom interactivo y ficha inferior de metadatos técnicos provenientes de PostgreSQL y Supabase Storage.



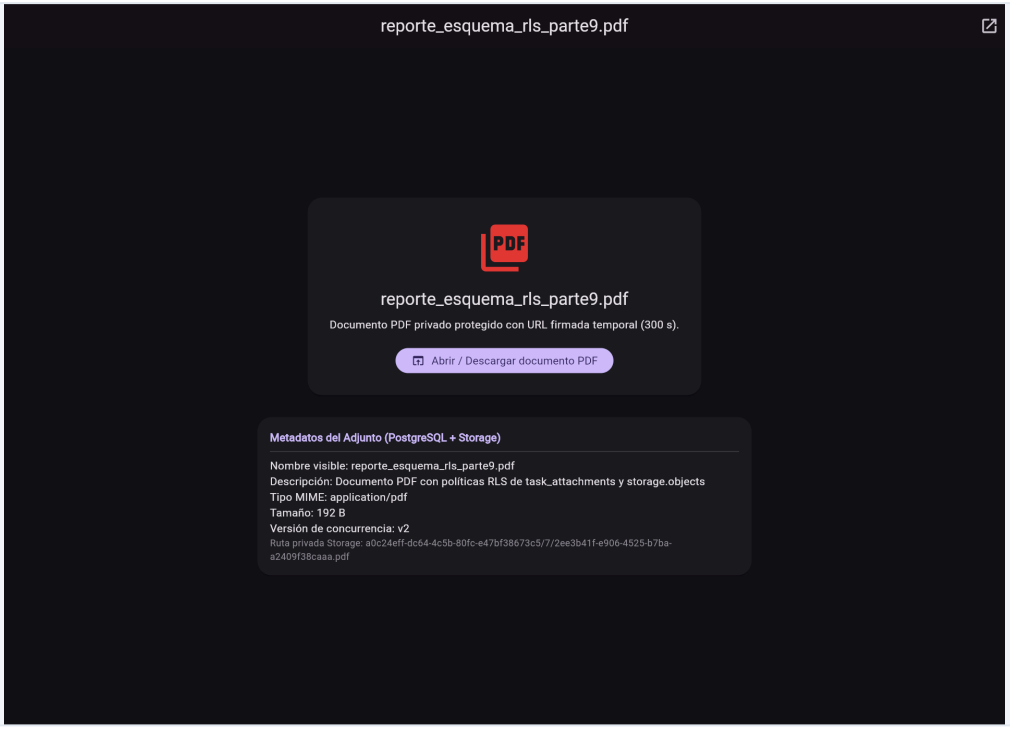
Evidencia 4 Visor Multimedia de Video MP4 (Versión v3 tras Reemplazo y Edición)

Reproducción del clip de video MP4 mostrando controles de reproducción, barra de progreso y metadatos actualizados a la **versión de concurrencia v3** tras reemplazo de archivo.



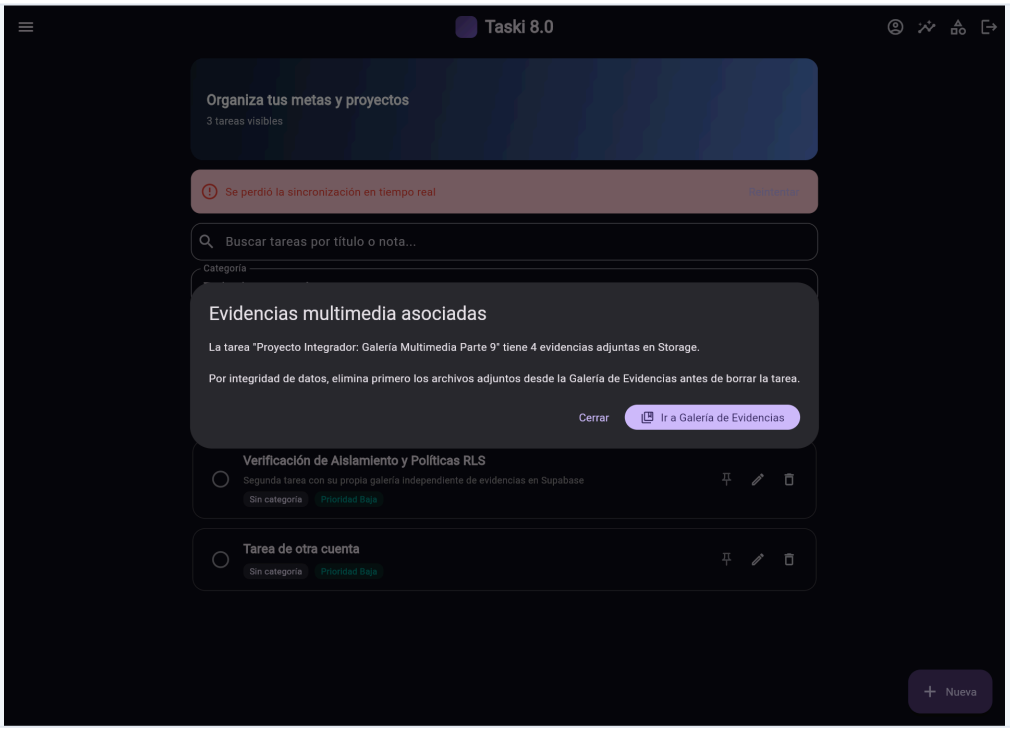
Evidencia 5 Visor de Documento PDF Protegido con URL Firmada Temporal (v2)

Vista de detalle del documento PDF con enlace firmado temporal de 300 segundos y descripción actualizada mediante `updateMetadata` con bloqueo optimista (v2).



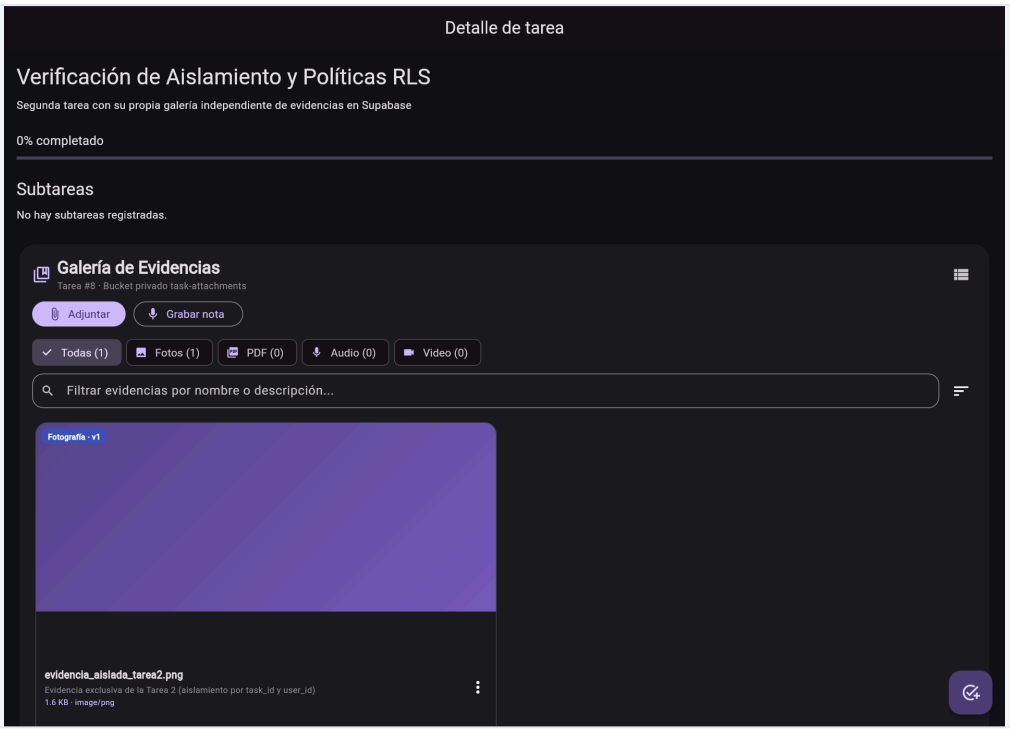
Evidencia 6 Protección de Integridad al Intentar Eliminar una Tarea con Evidencias (Sección 16)

Verificación previa en `HomeScreen` que detecta que la tarea posee 4 evidencias en Storage e impide su eliminación hasta limpiar primero la galería, evitando objetos huérfanos.



Evidencia 7 Segunda Tarea con su Propia Galería Independiente y Aislamiento RLS

Detalle de la Tarea #8 ("**Verificación de Aislamiento y Políticas RLS**") mostrando su galería aislada (`Todas (1)`, `Fotos (1)`) independiente de la Tarea #7 y protegida por `auth.uid() = user_id`.



Evidencia 8 — Ejecución de Pruebas Automatizadas (`flutter test`)

```
00:00 +0: Pruebas unitarias de TaskController (Parte 7) addTask agrega y recarga la lista
00:00 +1: Pruebas unitarias de TaskController (Parte 7) loadTasks expone un error cuando falla el repos
00:00 +5: Reto 7: Pruebas unitarias de validación de biografía (máx. 120 caracteres)
00:00 +6: TaskAttachment Model & TaskAttachmentService Validations (Parte 9) TaskAttachment.fromMap ma
00:00 +7: TaskAttachment Model & TaskAttachmentService Validations (Parte 9) inspectBytes rechaza archi
00:00 +8: TaskAttachment Model & TaskAttachmentService Validations (Parte 9) inspectBytes acepta los 4
00:03 +12: Pruebas de widget para LoginPage y ProfilePage
00:03 +13: All tests passed!
```